CSE470: Software Engineering

Repository, Client Server Architecture

What is a Repository

Before going into repository design pattern let's first understand what is repository:

- In generic terms repository simply means a place where things are or can be stored.
- This term has wide array of use in computer science for example code repository.
- We are all familiar with GITHUB. it’s a code repository

What is a Repository Architecture

- It will have multiple sub systems.
- Sub-systems must exchange data. This may be done in two ways:
  - Shared data is held in a central database or repository and may be accessed by all sub-systems;
  - Each sub-system maintains its own database and passes data explicitly to other sub-systems.

When large amounts of data are to be shared, the repository model of sharing is most commonly used as this is an efficient data sharing mechanism.

Formal Definition

A Repository mediates between the domain and data mapping layers, acting like an in-memory domain object collection. Client objects construct query specifications declaratively and submit them to Repository for satisfaction. Objects can be added to and removed from the Repository, as they can from a simple collection of objects, and the mapping code encapsulated by the Repository will carry out the appropriate operations behind the scenes.

Example

Let's think you are developing an IDE like ’eclipse’.What will you have there:

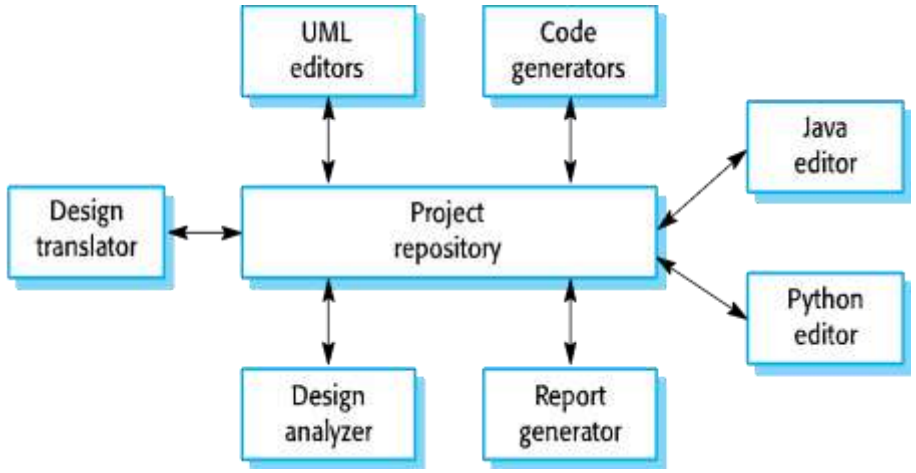
- editor for different language like java,c++.python
- code generator, auto complete system
- UML editor
- Translator to compile the code etc.

So here in the IDE where the components use a repository of system design information.

Each software tool generates information which is then available for use by other tools.

And, sub-components do not communicate with each other, means they are independent.

Have a look at the figure for the example



When Used

You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in **data-driven systems** where the inclusion of data in the repository triggers an action or tool.

Advantages-Disadvantages

Advantages

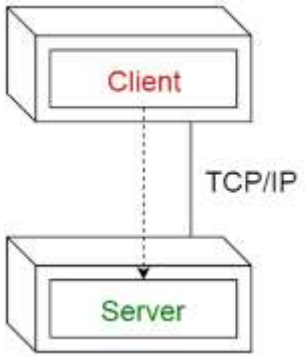
- Components can be independent—they do not need to know of the existence of other components.
- Changes made by one component can be propagated to all components.
- All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
- Data security is of higher importance.

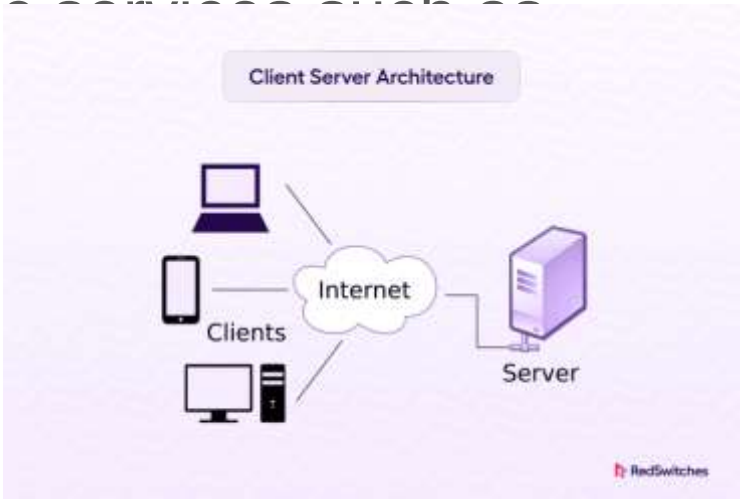
Disadvantages

- The repository is a single point of failure so problems in the repository affect the whole system.
- May be inefficiencies in organizing all communication through the repository – latency and processing overhead.
- Distributing the repository across several computers may be difficult. (Scalability issue, can be solved by synchronizing tools, but, complex and costly.)

What is Client-Server Architecture

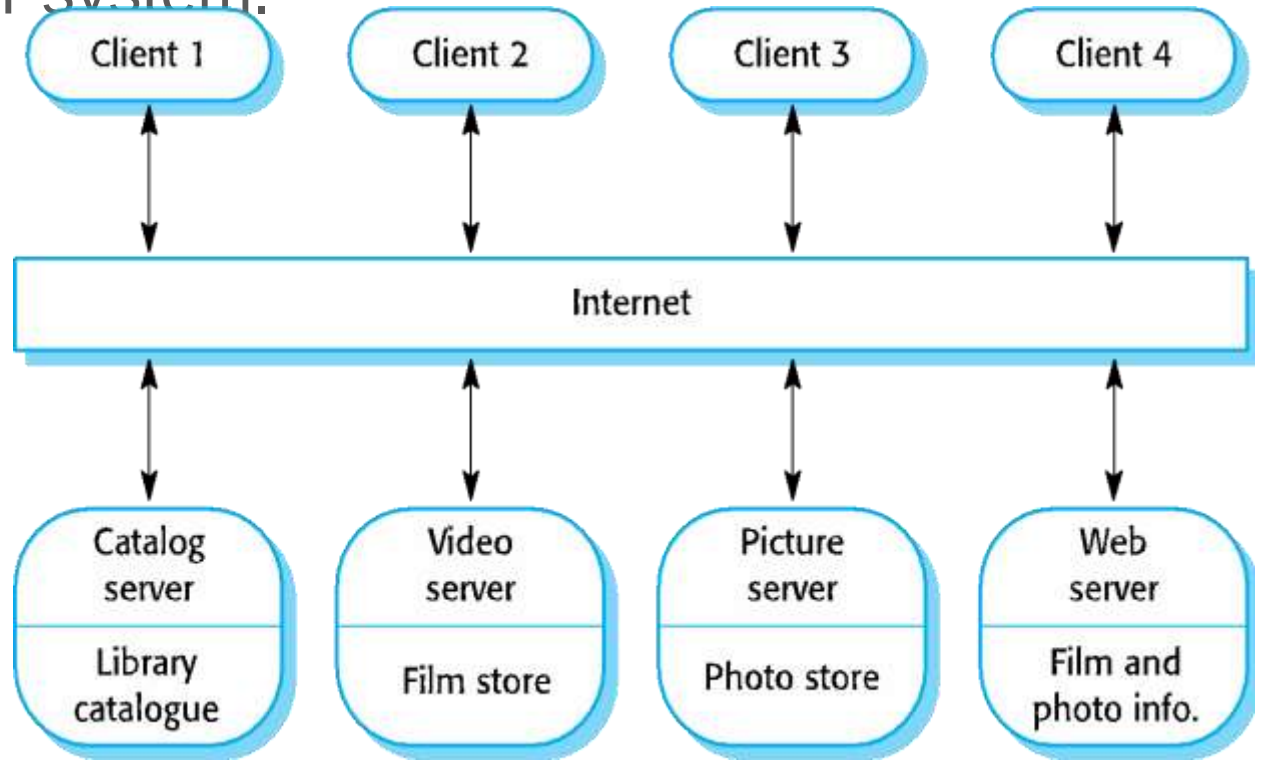
- This pattern consists of two parties; **a server and multiple clients**.
- The server component will provide services to multiple client components.
- Clients request services from the server and the server provides relevant services to those clients.
- Furthermore, the server continues to listen to client requests.
- Set of stand-alone servers which provide specifi
  printing, data management, etc.
- Set of clients which call on these services.
- Network which allows cli ess servers.





Example

- Online applications such as email, document sharing and banking.
- Figure is an example of a film and video/DVD library organized as a client–server system.



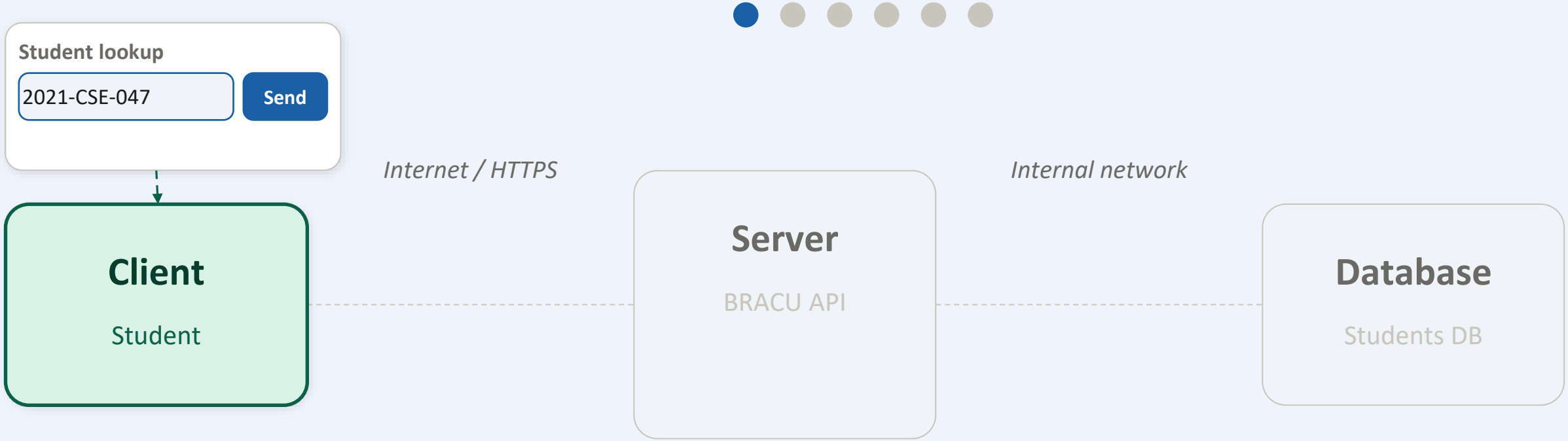
Client-Server Architecture

Student Record Request Flow

Browser → Server → Database → Response

Step 1/6

User enters student ID in the browser

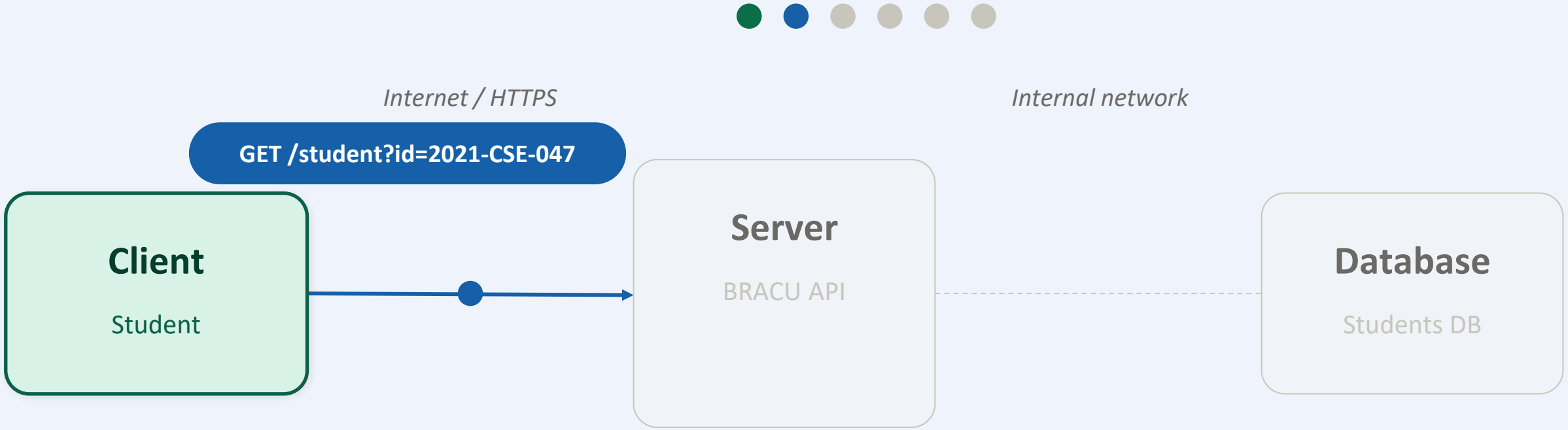


Network log

→ **REQUEST** User typed "2021-CSE-047" and clicked Send

Step 2/6

Browser sends HTTP request to server



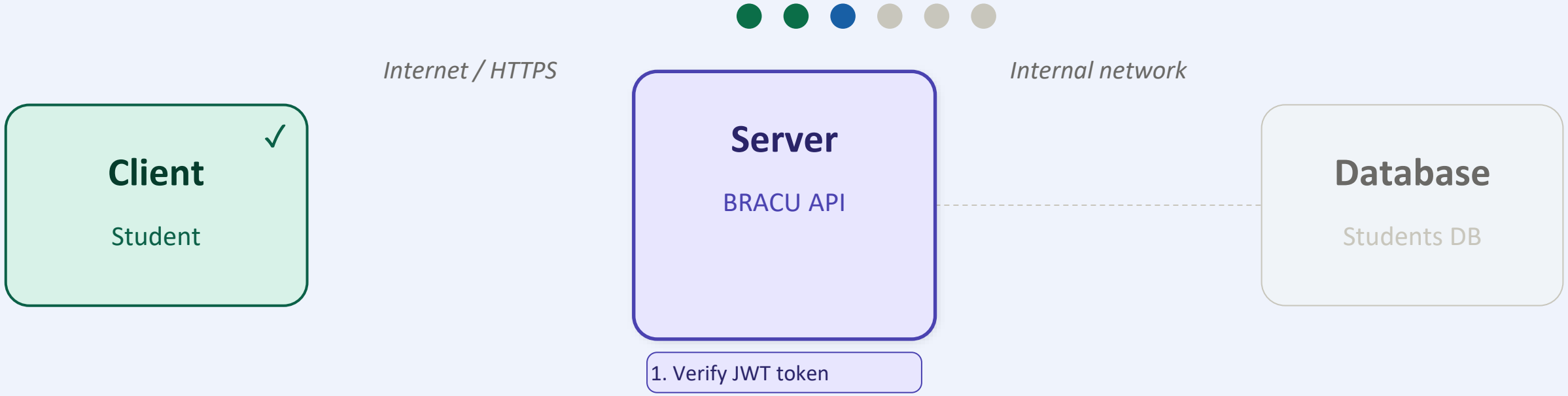
Network log

→ **REQUEST** GET /api/student?id=2021-CSE-047

→ **REQUEST** Headers: Authorization: Bearer <JWT>, Content-Type: application/json

Step 3/6

Server authenticates token and routes request



Network log

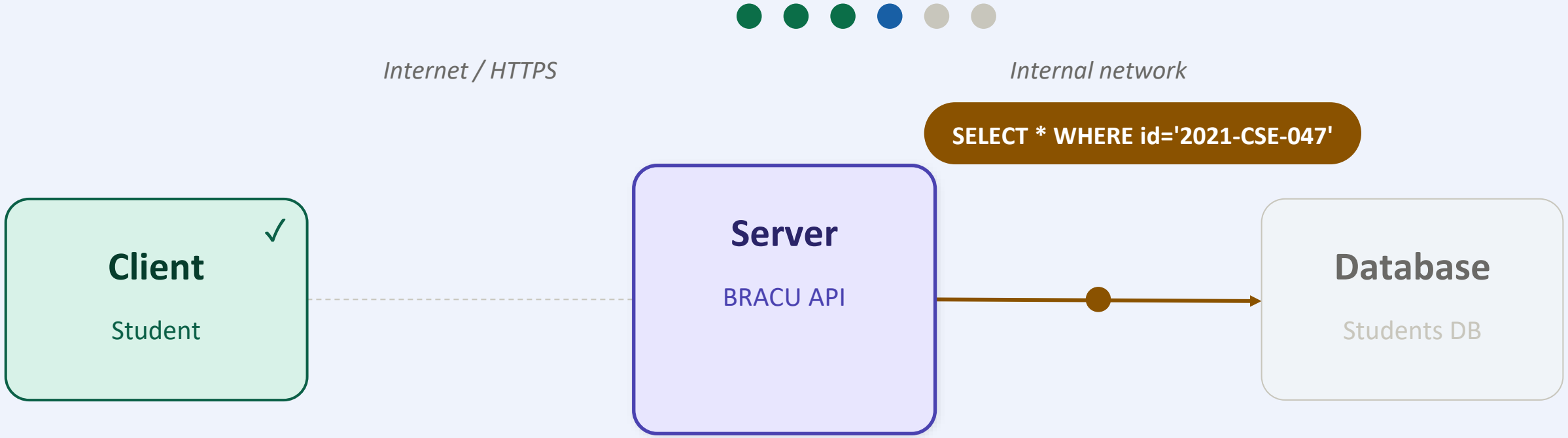
→ **REQUEST** GET /api/student?id=2021-CSE-047 received

SERVER JWT verified — user: admin@bracu.ac.bd

SERVER Routing to StudentController.getRecord()

Step 4/6

Server queries the database



Network log

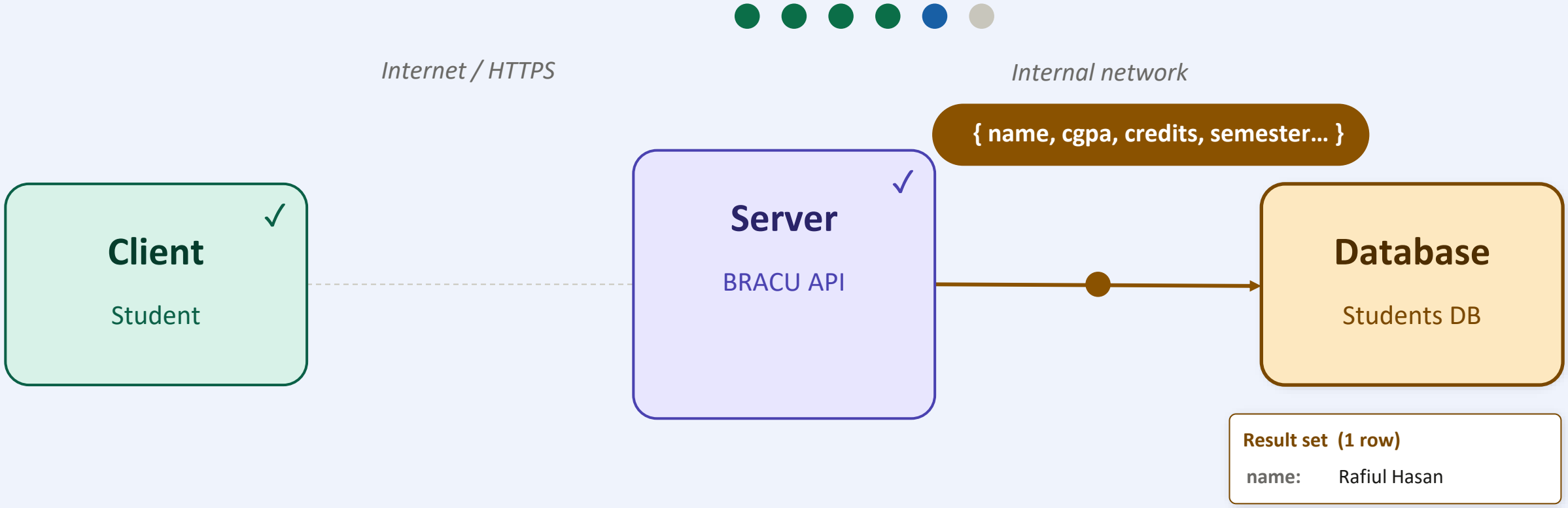
SERVER Building SQL query...

DB SELECT * FROM students WHERE student_id = '2021-CSE-047'

DB Query dispatched over internal network (not exposed to internet)

Step 5/6

Database returns the student record



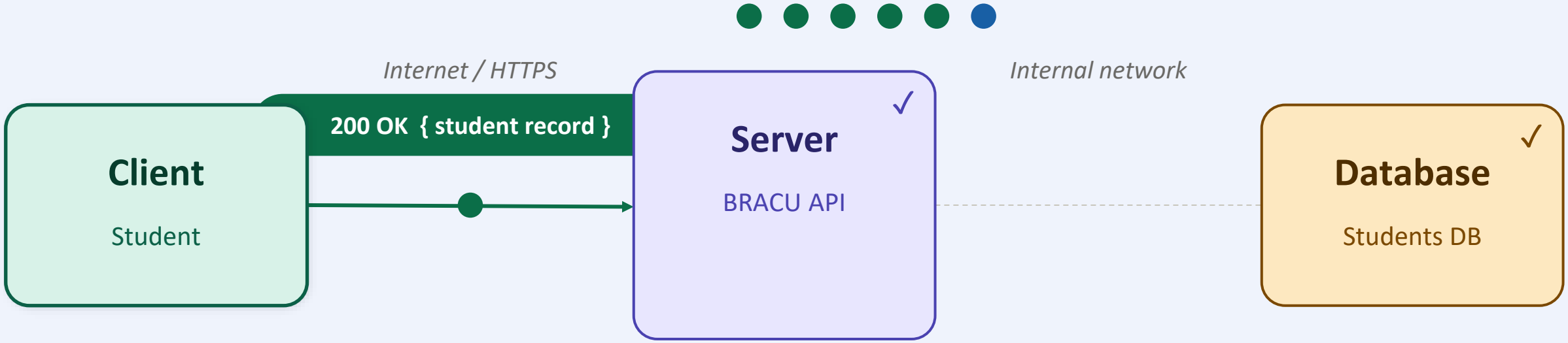
Network log

⬡ **DB** Query executed in 4ms — 1 row returned

⬡ **DB** { name: "Rafiul Hasan", cgpa: 3.72, credits: 120, semester: "8th" }

Step 6/6

Server delivers HTTP 200 response to browser



Network log

SERVER Serialising JSON — filtering sensitive fields, setting cache headers

✓ **RESPONSE** 200 OK — Content-Type: application/json — payload: 842 bytes

Response rendered — student record displayed

Internet / HTTPS

Client ✓

Student

RH

Rafiul Hasan

2021-CSE-047 · Computer Science & Engineering

CGPA	CREDITS EARNED
3.72 / 4.00	120

CURRENT SEMESTER	STATUS
8th	Active

Network log

✓ **RESPONSE** 200 OK — Student record received and rendered in 187ms total

✓ **RESPONSE** Browser updated — student dashboard now showing Rafiul Hasan's record

Full request lifecycle

1. User enters student ID
2. Browser sends HTTP request
3. Server authenticates & routes
4. Server queries the database
5. Database returns student record
6. 200 OK response to browser

When Used

It is used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable and can be resolved by a load balancer.

Advantages

- The principal advantage of this model is that servers can be distributed across a network.
- Data recovery is possible.
- Can be deployed to the cloud, hosted and managed by third-parties (AWS, Azure) which offloads physical infrastructure burden and easy to manage.

Disadvantages

- Each service is a single point of failure so susceptible to denial of service (DoS) attacks or server failure.
- Performance may be unpredictable because it depends on the network as well as the system.
- Cost inefficient as requires maintenance cost.
- May be management problems if servers are owned by different organizations.